

BÀI TẬP LẤY ĐIỂM

Bài 1: "Máy áp trứng" (Xử lý Chuỗi & Validate)

Đề bài:

- Màn hình A:** Có 1 ô nhập liệu EditText yêu cầu nhập Họ và Tên (Ví dụ: "nguyễn văn a").
- Yêu cầu logic:** Người dùng thường nhập ẩu (dư khoảng trắng, sai hoa thường). Trước khi chuyển trang, ứng dụng phải **"Tiền xử lý" (Pre-processing)** để chuẩn hóa tên thành dạng chuẩn ("Nguyễn Văn A").
- Màn hình B:** Nhận chuỗi đã chuẩn hóa và hiển thị: "Xin chào: Nguyễn Văn A". Đồng thời tách riêng Tên (A) và Họ (Nguyễn) để hiển thị ở 2 dòng khác nhau.

Điểm "lắc léo" cần giải quyết:

- Phải viết hàm chuanHoaTen(String input) riêng để xử lý logic trim(), lowercase(), và viết hoa chữ cái đầu.
- Phải viết hàm tachHoTen(String fullName) ở màn hình đích.

Gợi ý code xử lý (Cho sinh viên tham khảo):

Java

```
// Hàm tiền xử lý tại MainActivity
private String chuanHoaTen(String input) {
    if (input == null) return "";
    String result = "";
    // Xóa khoảng trắng thừa và tách từ
    String[] words = input.trim().replaceAll("\\s+", " ").toLowerCase().split(" ");

    for (String word : words) {
        if (word.length() > 0) {
            // Viết hoa chữ cái đầu
            result += Character.toUpperCase(word.charAt(0)) + word.substring(1) + " ";
        }
    }
    return result.trim();
}
```

// Lúc click nút Gửi

```
String rawName = edtName.getText().toString();
String cleanName = chuanHoaTen(rawName); // Tiền xử lý xong mới gửi
Intent intent = new Intent(this, ResultActivity.class);
intent.putExtra("CLEAN_NAME", cleanName);
startActivity(intent);
```

Bài 2: Bộ lọc số nguyên tố (Xử lý Mảng & Thuật toán)

Đề bài:

- **Màn hình A:** Nhập một dãy số bất kỳ ngăn cách bởi dấu phẩy (Ví dụ: "4, 7, 12, 11, 5, 20").
- **Màn hình B:** Chỉ nhận và hiển thị những số là **Số Nguyên Tố** trong dãy đó (Kết quả hiển thị: "7, 11, 5").
- **Yêu cầu kỹ thuật:** Chỉ được truyền 1 chuỗi String dài qua Intent. Màn hình B phải tự cắt chuỗi (split) và dùng hàm kiểm tra số nguyên tố để lọc.

Điểm "lắc léo":

1. Sinh viên phải xử lý chuyển đổi từ String -> String[] -> int.
2. Phải viết hàm kiểm tra số nguyên tố isPrime(int n) tách rời.

Gợi ý code logic tại Màn hình B (Nơi nhận):

Java

```
// Hàm kiểm tra số nguyên tố (Logic tách rời)
private boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i <= Math.sqrt(n); i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

```
// Trong onCreate của màn hình B
String dataReceive = intent.getStringExtra("DATA_NUMBER"); // Nhận "4, 7, 12..."
String[] arrStr = dataReceive.split(","); // Tách thành mảng
String ketQua = "";

for (String s : arrStr) {
```

```

try {
    int number = Integer.parseInt(s.trim()); // Phải trim để xóa khoảng trắng nếu có
    if (isPrime(number)) {
        ketQua += number + " "; // Cộng dồn kết quả
    }
} catch (NumberFormatException e) {
    // Xử lý nếu người dùng nhập bậy
}
}
txtResult.setText("Các số nguyên tố là: " + ketQua);

```

Bài 3: Giải phương trình & Biện luận (Logic điều kiện)

Đề bài:

- **Màn hình A:** Nhập 3 số a, b, c.
- **Hành động:** Nhấn nút "Giải Phương Trình".
- **Màn hình B:**
 - Hiển thị kết quả nghiệm.
 - **Tuy nhiên:** Nếu $a = 0$ (trở thành phương trình bậc 1), hoặc $\Delta < 0$ (vô nghiệm), phải hiển thị thông báo lỗi cụ thể và đổi màu chữ (Ví dụ: Vô nghiệm thì chữ đỏ, Có nghiệm thì chữ xanh).

Điểm "lắc léo":

- Thay vì tính toán hết ở A rồi gửi kết quả String sang B. Hãy bắt sinh viên **gửi a, b, c (dạng double)** sang B.
- Màn hình B phải gánh toàn bộ logic toán học (Math.pow , Math.sqrt) và logic hiển thị (UI).

Mở rộng (Khó hơn):

Yêu cầu sinh viên viết 1 class MathHelper.java (chỉ chứa các hàm static, không phải model object) để gọi hàm tính Delta.

Java

```

// Tại Màn hình B
double a = intent.getDoubleExtra("VAL_A", 0);
double b = intent.getDoubleExtra("VAL_B", 0);
double c = intent.getDoubleExtra("VAL_C", 0);

```

```
// Logic biện luận nằm ở đây
if (a == 0) {
    // Giải pt bậc 1: bx + c = 0
    if (b == 0) {
        txtResult.setText(c == 0 ? "Vô số nghiệm" : "Vô nghiệm");
    } else {
        txtResult.setText("Nghiệm x = " + (-c / b));
    }
} else {
    // Giải pt bậc 2...
}
```

Bài 4: Ứng dụng "Mật thư" (Mã hóa Caesar đơn giản)

- **Mô tả:**
 - **Màn hình A:** Nhập một câu thông điệp (Ví dụ: "abc"). Nhập một số nguyên K (Key - số bước nhảy).
 - **Hành động:** Nhấn nút "Mã hóa & Gửi".
 - **Logic (Tiền xử lý tại A):** Trước khi gửi, viết hàm dịch chuyển từng ký tự trong chuỗi sang K đơn vị trong bảng mã ASCII (Ví dụ: 'a' + 1 = 'b').
 - **Màn hình B:** Chỉ nhận và hiển thị chuỗi đã bị mã hóa (Ví dụ: "bcd").
- **Điểm lắt léo:** Sinh viên phải làm việc với char, ép kiểu (int) sang (char) và xử lý vòng lặp chuỗi.
- **Gợi ý code (Hàm xử lý):**

```
Java
// Input: "abc", k=1 -> Output: "bcd"
public String caesarCipher(String text, int k) {
    StringBuilder result = new StringBuilder();
    for (char character : text.toCharArray()) {
        result.append((char)(character + k));
    }
    return result.toString();
}
```

Bài 5: Tính chỉ số BMI & Lời khuyên sức khỏe

- **Mô tả:**
 - **Màn hình A:** Nhập Chiều cao (cm) và Cân nặng (kg).
 - **Màn hình B:** Hiển thị chỉ số BMI và **Câu kết luận**.
- **Logic (Xử lý tại B):**

- Nhận dữ liệu. Đổi cm sang m. Tính BMI = CanNang / (ChieuCao * ChieuCao).
- **Quan trọng:** Dựa vào kết quả BMI để hiển thị màu sắc và text khác nhau (BMI < 18.5: Gầy (Chữ Vàng), 18.5-24.9: Bình thường (Chữ Xanh), >25: Thừa cân (Chữ Đỏ)).
- **Điểm lắt léo:** Xử lý if-else if-else lồng nhau và thay đổi thuộc tính giao diện (setTextColor) bằng code Java dựa trên logic.

Bài 6: Máy tính tiền điện (Logic bậc thang)

- **Mô tả:**
 - **Màn hình A:** Nhập số điện tiêu thụ (kWh).
 - **Màn hình B:** Hiển thị tổng số tiền phải trả.
- **Logic (Bài toán kinh điển):** Tính tiền theo bậc thang (50 số đầu giá 1.678đ, 50 số sau giá 1.734đ...).
- **Điểm lắt léo:** Sinh viên thường tính sai logic lũy tiến (nhân thẳng số tổng với giá cao nhất là sai). Bài này bắt buộc phải viết một hàm tinhTienDien(int soDien) riêng biệt, cẩn thận.

Bài 7: Kiểm tra tính hợp lệ của Tam Giác

- **Mô tả:**
 - **Màn hình A:** Nhập 3 cạnh a, b, c.
 - **Màn hình B:** Kết luận đây là tam giác gì? (Thường, Cân, Đều, Vuông hay Không phải tam giác).
- **Logic:**
 1. Check điều kiện tồn tại: $(a+b>c) \ \&\& \ (a+c>b) \ \&\& \ (b+c>a)$. Nếu sai -> "Không phải tam giác" -> Kết thúc.
 2. Check Tam giác đều $(a=b=c)$.
 3. Check Tam giác vuông (Pytago: $a^2 = b^2 + c^2$...).
- **Điểm lắt léo:** Thứ tự kiểm tra if. Nếu check "Cân" trước "Đều" có thể bị sai logic (vì đều cũng là cân). Phải check trường hợp đặc biệt nhất (Đều) trước.

Bài 8: Đổi đơn vị tiền tệ (Real-time formatting)

- **Mô tả:**
 - **Màn hình A:** Nhập số tiền VND (Ví dụ nhập: 10000000).
 - **Màn hình B:** Hiển thị số tiền quy đổi ra USD, EUR, JPY.
- **Logic:**
 - Yêu cầu định dạng số (Format Currency): Kết quả không được hiện "434.782608..." mà phải hiện "\$434.78" (làm tròn 2 số thập phân).
- **Điểm lắt léo:** Sử dụng DecimalFormat hoặc String.format("%.2f", value) để hiển thị đẹp mắt. Đây là kỹ năng rất cần khi làm App thực tế.

Bài 9: Tìm số lớn nhất, nhỏ nhất (Truyền Mảng)

- **Mô tả:**
 - **Màn hình A:** Nhập 4 số nguyên vào 4 ô EditText khác nhau.

- **Màn hình B:** Hiển thị số Max và số Min.
- **Cách truyền:** Thay vì putExtra 4 lần (so1, so2, so3, so4). Yêu cầu sinh viên gom 4 số này vào một mảng int[] hoặc ArrayList<Integer> rồi truyền sang B bằng putIntegerArrayListExtra.
- **Logic tại B:** Viết thuật toán tìm Max/Min từ mảng nhận được.
- **Điểm lắt léo:** Làm quen với việc truyền Collection (List/Array) qua Intent thay vì biến đơn.

Bài 10: Ứng dụng "Bói tuổi" (Xử lý Ngày tháng - Date Logic)

- **Mô tả:**
 - **Màn hình A:** Nhập Năm sinh (Ví dụ: 1990).
 - **Màn hình B:**
 1. Tính tuổi hiện tại.
 2. Cho biết thuộc thế hệ nào (Gen Z, Gen Y, Gen X...).
 3. Kiểm tra năm đó có phải năm nhuận không?
 - **Điểm lắt léo:**
 - Lấy năm hiện tại động (Dynamic): Không được hard-code 2025 - nămSinh. Phải dùng Calendar.getInstance().get(Calendar.YEAR) để code chạy đúng mãi mãi về sau.
 - Logic năm nhuận: $(\text{nam} \% 4 == 0 \ \&\& \ \text{nam} \% 100 != 0) \ || \ (\text{nam} \% 400 == 0)$.
-